#08/01/2025

Unlock the power of Hexagonal Architecture

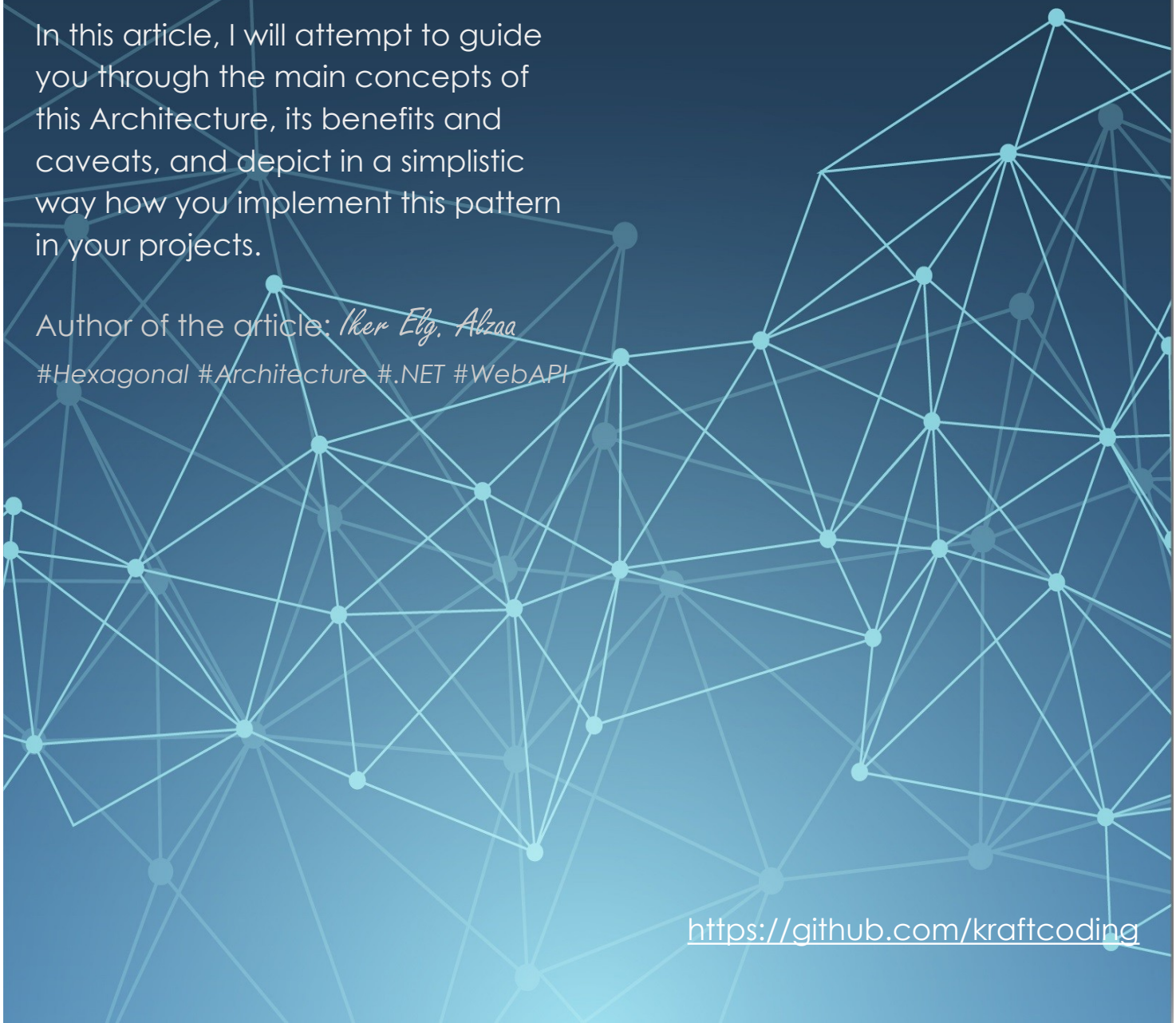
Source code here

In C#, we use a wide range of architectures, designs, and patterns. One of the most popular is therefore Hexagonal Architecture, which we also call ports and adapters due to its structure.

In this article, I will attempt to guide you through the main concepts of this Architecture, its benefits and caveats, and depict in a simplistic way how you implement this pattern in your projects.

Author of the article: *Iker Elg, Alzaa*

#Hexagonal #Architecture #.NET #WebAPI



https://github.com/kraftcoding

The *Hexagonal Architecture*, also referred to as *Ports* and *Adapters*, is an architectural pattern that allows input by users or external systems to arrive into the Application at a Port via an Adapter, and allows output to be sent out from the application through a Port to an Adapter.

This creates an abstraction layer that protects the core of an application and isolates it from external tools and technologies. This is a great advancement, since it does not depend on external factors, testing it becomes natural and mocking its dependencies is easy.

It also lets you design all your system's interfaces 'by purpose' rather than by technology, preventing you from lock-in, and making it easier for your application's tech stack to evolve with time.

If you need to change the persistence layer, go for it. All you need to do is implement new Adapters and you're good to go.

Ports

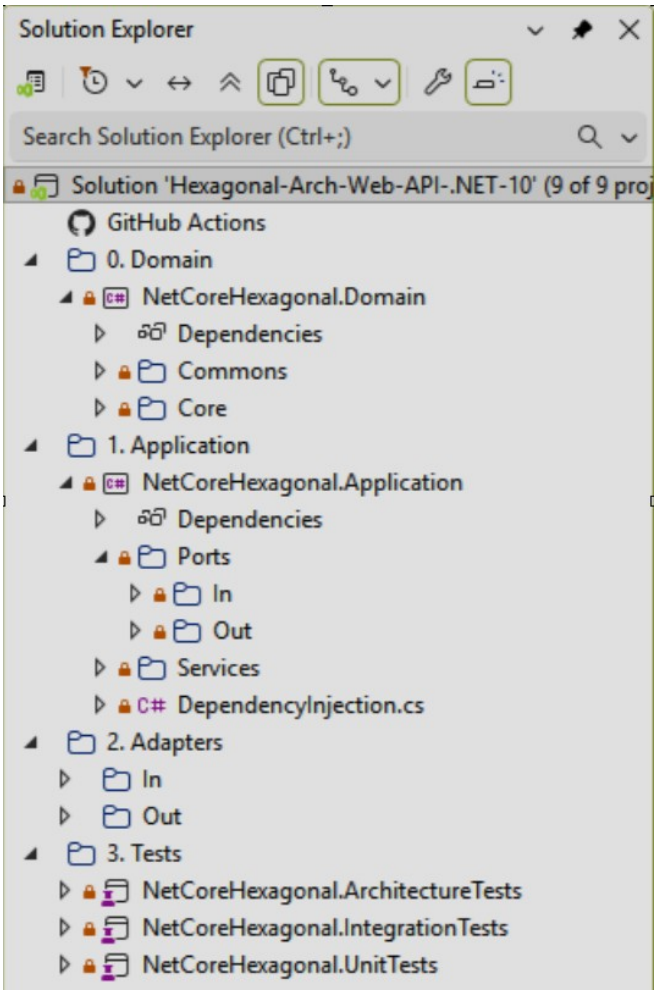
We can see a Port as a technology-agnostic entry point, it determines the interface which will allow foreign actors to communicate with the Application, regardless of who or what will implement said interface. Ports also allow the Application to communicate with external systems or services, such as databases, message brokers, other applications, etc. A Port should always have two items hooked to it, one always being a test.

Adapters

An Adapter will initiate the interaction with the Application through a Port, using a specific technology, for example, a REST controller would represent an adapter that allows a client to communicate with the Application. There can be as many Adapters for any single Port as needed without this representing a risk to the Ports or the Application itself.

Application

The Application is the core of the system, it contains the Application Services which orchestrate the functionality or the use cases.



Domain Layer

The Ports and Adapters architecture also plays along very well with Domain-Driven Design, the main advantage it brings is that it shields domain logic to leak out of your application's core.

When paired with Domain-Driven Design, the Application, or Hexagon, contains both the Application and the Domain layers, as it is the

case of the provided source code, leaving the User Interface and Infrastructure layers outside.

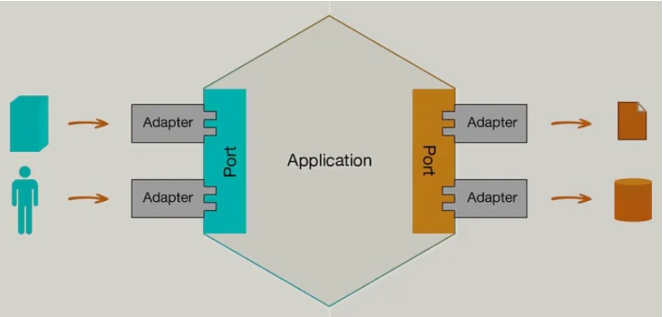
Otherwise, It will contain the Domain Model together with the business logic embedded in Aggregates.

Actors

The actors that act upon this distribution of logic are the following.

- *Driving (or primary)* actors are the ones that initiate the interaction, and are always depicted on the left side. For example, a driving adapter could be a controller which is the one that takes the (user) input and passes it to the Application via a Port.
- *Driven (or secondary)* actors are the ones that are "kicked into behaviour" by the Application. For example, a database Adapter is called by the Application so that it fetches a certain data set from persistence.

Here an image that illustrates what we mentioned before.



At this point, to understand better what we have seen so far, I am going to show you how to implement all needed components in the image.

Implementation

In this section, we will implement a highly simplified version of a book registration service that processes requests to create book registers in the data base.

But first some clarifications.

When it comes to implementation, there are some important details that shouldn't be missed:

- Ports will be (most of the time, depending on the language you choose) represented as interfaces in code.
- Driving Adapters will use a Port and an Application Service that will implement by an Interface located in the application layer.
- Driven adapters will implement the Port and the *Application Service* will use it.
- It's important to highlight that all layers of Domain-Driven Design's Layered Architecture are still suitable when structuring an application based on Ports and Adapters, as they provide an ideal segregation for all components.

As mentioned before, the left and right sides of the Hexagon contain two different types of actors, Driving and Driven where both Ports and Adapters exist.

- *On the Driving side*, the Adapter depends on the Port, which is implemented by the Application Service, therefore the Adapter doesn't know who will react to it's invocations, it just knows what methods are guaranteed to be available, therefore it depends on an abstraction.

- *On the Driven side*, the Application Service is the one that depends on the Port, and the Adapter is the one that implements the Port's Interface, effectively inverting the dependency since the 'low-level' adapter (i.e. database repository) is forced to implement the abstraction defined in the application's core, which is 'higher-level'.

So, let's start with the code implementation.

Please note that the examples intentionally omit certain parts and disregard important aspects of well-written code, such as error handling and proper naming, to focus on the basic concepts of implementing Ports and Adapters.

Adapter WebAPI In

We will add the controller function to the adapter that in this case is a Web API.

```
[HttpPost("books")]
0 references | 0 changes | 0 authors, 0 changes
public async Task<ActionResult> RegisterBook([FromBody] RegisterBookDto dto)
{
    logger.LogInformation($"Registering {dto}...");

    var courseDto = await schoolService.RegisterBook(dto);

    return Ok(courseDto);
}
```

Application Service In

Then, we will add the required modifications to the *Application Service (SchoolService.cs)*.

Application Ports In and Dtos

And add the required ports *(IschoolService.cs)* and the associated *Dtos (CourseDto.cs,*

```
namespace NetCoreHexagonal.Application.Ports.In
{
    7 references | kraftcoding, 1 day ago | 1 author, 1 change
    public interface ISchoolService
    {
        2 references | kraftcoding, 1 day ago | 1 author, 1 change
        Task<IReadOnlyList<CourseDto>> GetAllCourses();
        2 references | kraftcoding, 1 day ago | 1 author, 1 change
        Task<IReadOnlyList<StudentDto>> GetAllStudents();
        5 references | kraftcoding, 1 day ago | 1 author, 1 change
        Task<CourseDto> RegisterCourse(RegisterCourseDto dto);
        4 references | kraftcoding, 1 day ago | 1 author, 1 change
        Task<StudentDto?> RegisterStudent(RegisterStudentDto dto);
        4 references | kraftcoding, 1 day ago | 1 author, 1 change
        Task EnrollStudent(EnrollStudentDto dto);
        1 reference | 0 changes | 0 authors, 0 changes
        Task<BookDto> RegisterBook(RegisterBookDto dto);
    }
}
```

RegisterCourseDto.cs) too, due to clear separation between *Entities and Data Transfer Object* is needed.

Domain Core

As in this case we separated the domain layer, the needed classes for that layer must be added too to the project strucure *(Books.cs, BooksName.cs)*.

Application Ports out

To interface with the repository, we need to create the needed classes (*IbookRepository.cs*).

Adapter Persistence Service out

Finally, we need to update the persistence service classes (*BookRepository.cs, SchoolDBContext.cs*).